**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ**

ФГБОУ ВО «Брянский государственный технический университет»

Факультет информационных технологий

Кафедра «Информатика и программное обеспечение»

# Отчёт по учебной практике

**по теме**

**«Проектирование гитарного плагина GuitarToolkit»**

Выполнил: студент гр. О-23-ИВТ-2-ПО-Б

Аникьев Е. Ю.

_____

*Подпись студента*

Проверил: _____

_____

«___» _____ 20__ г.

_____

*Подпись преподавателя*

Брянск 2026

# Оглавление

Введение ..... 3
1. Теоретическая часть ..... 4
1.1. Анализ предметной области ..... 4
1.2. Вывод ..... 5
2. Конструкторская часть ..... 6
2.1. Архитектура классов ..... 6
2.2. Основные структуры данных ..... 8
2.3. Основные функции ..... 10
3. Экспериментальная часть ..... 14
3.1. Руководство пользователя ..... 14
3.2. Тестирование ..... 14
3.2.1. Первый тест ..... 15
3.2.2. Второй тест ..... 16
3.2.3. Третий тест ..... 17
3.2.4. Выводы ..... 18
Заключение ..... 19
Список литературы ..... 20

# ВВЕДЕНИЕ

Учебная практика является важным этапом закрепления теоретических знаний и получения практических навыков проектирования программных продуктов. В рамках данной работы рассматривался процесс разработки программного продукта GuitarToolkit, предназначенного для гитаристов и музыкантов, использующих персональный компьютер как инструмент обучения, настройки инструмента и работы в домашней студии.

Темой практической работы является проектирование гитарного плагина GuitarToolkit. Данный программный продукт объединяет функции, которые обычно реализуются в нескольких самостоятельных приложениях: тюнер, метроном, справочник аккордов, визуализатор гамм, тренажёр интервалов, построитель аккордовых прогрессий и круг квинт.

Актуальность разработки обусловлена тем, что музыканту часто требуется единая среда, позволяющая быстро настроить инструмент, отработать ритм, изучить аппликатуры аккордов и проверить музыкально-теоретические построения без переключения между разными программами. Дополнительную ценность представляет возможность работы как в standalone-режиме, так и внутри DAW-хоста в формате VST3.

В ходе практики была изучена предметная область, сформулированы требования к программному продукту, определена архитектура решения, реализованы основные функциональные модули, выполнены сборка, тестирование и подготовка проекта к публикации. Отчёт оформлен на основе технического задания и фактической структуры проекта GuitarToolkit.

# 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

## 1.1. Анализ предметной области

Предметной областью проекта является разработка программного инструмента для гитаристов, совмещающего функции музыкального справочника, тренировочного средства и аудиоплагина. В процессе обучения и домашней записи музыкант постоянно обращается к нескольким типам данных: нотам, частотам, строям, аккордам, гаммам, интервалам, тональностям и ритмическим параметрам.

На практике эти задачи часто решаются разными приложениями. Для настройки инструмента используется тюнер, для работы с ритмом - метроном, для изучения гармонии - справочники аккордов, таблицы гамм и круг квинт. Такой подход неудобен, потому что пользователь вынужден переключаться между программами и теряет единый контекст работы.

GuitarToolkit проектируется как единая среда, в которой базовые задачи гитариста доступны через один интерфейс. Приложение должно быть полезно как начинающему пользователю, изучающему аппликатуры и интервалы, так и музыканту, который работает в цифровой звуковой рабочей станции и хочет использовать тюнер или метроном прямо в проекте записи.

Особенностью предметной области является связь музыкальной теории с цифровой обработкой звука. Нота может рассматриваться как элемент музыкальной системы, но в тюнере она также соответствует конкретной частоте. Аккорд описывается формулой интервалов, но в интерфейсе должен быть представлен в виде аппликатуры на гитарном грифе. Поэтому проект требует сочетания теоретической модели и прикладных алгоритмов.

Основными сущностями предметной области являются:

- нота и частота, связанные равномерно темперированным строем;
- строй гитары, определяющий базовые частоты открытых струн;
- аккорд, включающий тонику, тип, интервальную формулу и аппликатуру;
- гамма, задаваемая тоникой, ладом и последовательностью ступеней;

• интервал, используемый для тренировки слуха и анализа гармонии;
• аудиопоток, поступающий от микрофона, аудиоинтерфейса или DAW-хоста.

С точки зрения пользователя важна не только корректность расчётов, но и наглядность. Гитарист воспринимает музыкальный материал через расположение нот на грифе, визуальные схемы аккордов, подсветку активных элементов и быстрый отклик интерфейса. Поэтому интерфейс GuitarToolkit построен вокруг вкладок, каждая из которых соответствует отдельному музыкальному инструменту.

Для работы внутри DAW проект должен учитывать ограничения аудиоплагина. Плагин вызывается хостом в реальном времени, поэтому обработчик аудио не должен выполнять тяжёлые операции, блокировать поток или обращаться к файлам в критическом участке. Устойчивость VST3-режима является важным требованием, так как ошибка плагина способна привести к аварийному завершению всего хоста.

## 1.2. Вывод

Проведённый анализ показывает, что GuitarToolkit целесообразно проектировать как модульное приложение с общим ядром музыкальной логики и двумя формами поставки: standalone-приложением для Windows и VST3-плагином для DAW. Такой подход позволяет сохранить единый пользовательский опыт и одновременно поддержать разные сценарии использования.

Теоретическая часть проекта опирается на модели музыкальной теории, а практическая часть требует алгоритмов цифровой обработки аудио и аккуратной организации пользовательского интерфейса. Следовательно, дальнейшее проектирование должно разделить предметную логику, визуальные компоненты и платформенные механизмы работы со звуком.

# 2. КОНСТРУКТОРСКАЯ ЧАСТЬ

## 2.1. Архитектура классов

Архитектура GuitarToolkit построена по принципу разделения ответственности. Решение GuitarToolkit.sln включает несколько проектов, каждый из которых отвечает за отдельный слой приложения. Такое устройство позволяет независимо развивать ядро, пользовательский интерфейс, desktop-оболочку, VST3-плагин и тестовый набор.

Проект GuitarToolkit.Core содержит модели музыкальной теории, DSP-алгоритмы и сервисы. Он не зависит от WPF, NAudio и AudioPlugSharp, поэтому может использоваться в разных режимах запуска и проверяться модульными тестами без открытия графического интерфейса.

Проект GuitarToolkit.UI содержит общие WPF-компоненты: тюнер, метроном, справочник аккордов, визуализатор грифа, тренажёр интервалов, построитель прогрессий и круг квинт. Этот проект используется как desktop-приложением, так и VST3-плагином, что уменьшает дублирование кода.

Проект GuitarToolkit.Desktop реализует самостоятельное Windows-приложение. Он отвечает за запуск главного окна, выбор устройства ввода, захват звука через NAudio и воспроизведение синтезированных сигналов. Проект GuitarToolkit.Plugin является точкой входа VST3-плагина и получает аудиобуферы от DAW-хоста через AudioPlugSharp.

На рисунке 1 представлена обобщённая схема программного продукта GuitarToolkit. Она показывает связь между пользователем, интерфейсом, ядром приложения, DSP-алгоритмами, хранением данных и платформенными аудиохостами.

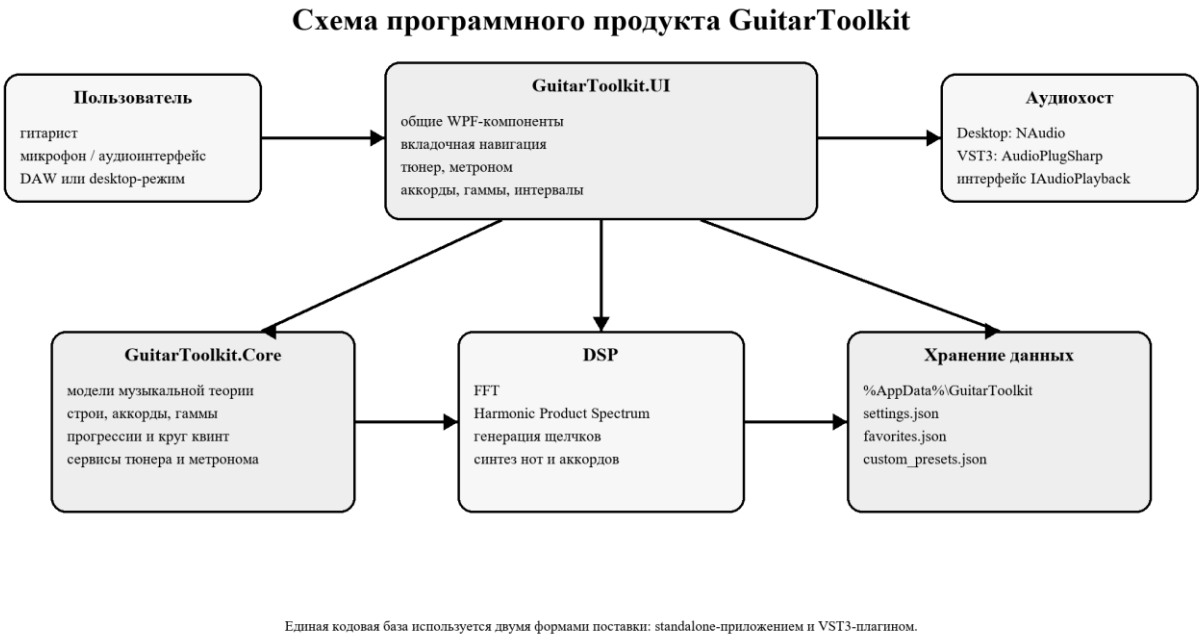


*Рис. 1. Концептуальная схема программного продукта GuitarToolkit.*

Схема показывает, что пользовательский интерфейс не зависит напрямую от конкретного режима запуска. В desktop-режиме аудиоввод и аудиовывод обеспечиваются средствами NAudio, а в режиме VST3 аудиобуферы поступают от DAW-хоста через AudioPlugSharp.

Ключевым элементом архитектуры является общее ядро. В нём сосредоточены классы PitchDetector, TunerEngine, MetronomeEngine, модели нот, гамм, аккордов, интервалов и прогрессий. Благодаря этому разные оболочки используют одну и ту же логику, а изменения в музыкальных алгоритмах не требуют переписывания интерфейса.

Для воспроизведения звуков используется абстракция IAudioPlayback. Она скрывает различия между desktop-режимом и VST3-режимом. Компонент интерфейса вызывает единый метод воспроизведения, а конкретная реализация уже решает, как добавить сигнал в выходной поток.

**Листинг 1**

Интерфейс воспроизведения аудио, общий для desktop-версии и VST3-плагина

```
public interface IAudioPlayback
{
```

```
        int SampleRate { get; }
        void PlaySamples(float[] samples);
        void StopPlayback();
    }
```

Интерфейс IAudioPlayback позволяет компонентам пользовательского интерфейса запускать воспроизведение нот, аккордов и гамм без знания конкретного аудиохоста. В desktop-режиме вызов попадает в реализацию на NAudio, а в VST3-режиме - в реализацию плагина.

## 2.2. Основные структуры данных

Основные структуры данных проекта отражают предметную область. Ноты представлены в хроматической системе, где каждая нота имеет имя и может быть связана с частотой. Пересчёт частоты в ноту выполняется на основе равномерно темперированного строя и эталонной частоты A4.

Строй гитары задаётся набором открытых струн. Для стандартной шестиструнной гитары используются значения E2, A2, D3, G3, B3, E4. В тюнере также предусмотрены альтернативные строи, например Half Step Down, Drop D и другие варианты, необходимые музыкантам в реальной практике.

Аккорд описывается тоникой, типом, интервальной формулой и набором аппликатур. Аппликатура содержит информацию о ладах и струнах, благодаря чему может быть отображена на диаграмме грифа. Дополнительно хранится сложность аккорда и возможность добавления в избранное.

Гамма задаётся тоникой и интервальным шаблоном. На основе этих данных вычисляются ноты гаммы, после чего они отображаются на виртуальном грифе. Пользователь может переключать режим отображения между названиями нот и ступенями.

Пользовательские данные сохраняются в каталоге %AppData%\GuitarToolkit. Настройки приложения хранятся в файле settings.json, избранные аккорды - в favorites.json, пользовательские пресеты прогрессий - в custom_presets.json. JSON выбран как простой и наглядный формат, удобный для небольших пользовательских данных.

В ядре приложения выделены следующие группы структур и сервисов:

- DSP - реализация FFT, Harmonic Product Spectrum, генерации щелчков и синтеза нот;
- Models - модели нот, строев, аккордов, гамм, интервалов, прогрессий и круга квинт;
- Services - движки тюнера и метронома, сохранение настроек и работа с пользовательскими пресетами;
- UI Components - визуальные вкладки, связывающие пользовательские действия с сервисами ядра.

**Листинг 2**

Структура данных для описания аппликатуры аккорда

```
public enum ChordDifficulty { Easy, Medium, Hard }

public record ChordDefinition(
    string Root,
    string Type,
    int[] Frets,
    int BaseFret)
{
    public string DisplayName => Root + Type;

    public ChordDifficulty Difficulty
    {
        get
        {
            bool hasOpenStrings = Frets.Any(f => f == 0);
            bool hasBarre = BaseFret > 1;
            int maxFret = Frets.Where(f => f > 0).DefaultIfEmpty(0).Max();
            int span = maxFret - Frets.Where(f => f > 0).DefaultIfEmpty(0).Min();

            if (hasOpenStrings && !hasBarre && maxFret <= 3)
                return ChordDifficulty.Easy;
            if (span >= 4 || maxFret >= 10)
                return ChordDifficulty.Hard;
            return ChordDifficulty.Medium;
        }
    }
}
```

Класс ChordDefinition хранит музыкальное и визуальное описание аккорда. Массив Frets содержит шесть значений для струн гитары: -1 обозначает заглушенную струну, 0 - открытую струну, положительное число - лад. На основе этих данных строится диаграмма аппликатуры.

**Листинг 3**

Сохранение избранных аккордов пользователя

```
private static string FavoritesPath =>
    Path.Combine(
        Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData),
        "GuitarToolkit", "favorites.json");
```

```
public static void ToggleFavorite(string root, string type)
{
    string key = $"{root}|{type}";
    if (!Favorites.Remove(key))
        Favorites.Add(key);
    SaveFavorites();
}
```

Избранные аккорды сохраняются в пользовательском каталоге приложения. Такой способ хранения не требует базы данных и подходит для небольшого набора пользовательских настроек.

## 2.3. Основные функции

Модуль тюнера выполняет анализ входного аудиосигнала, определяет частоту основного тона, сопоставляет её с ближайшей нотой и отображает отклонение в центах. Для устойчивости результата используется RMS-порог тишины, сглаживание частоты, FFT, Harmonic Product Spectrum и параболическая интерполяция пика спектра.

Модуль метронома предназначен для тренировки ритма. Он поддерживает диапазон 30-300 BPM, выбор размера такта, акцентирование первой доли и регулировку громкости. Важной особенностью является генерация щелчков по позиции сэмплов, что повышает точность по сравнению с UI-таймером.

Справочник аккордов предоставляет выбор тоники, типа аккорда и фильтров сложности. Пользователь видит название аккорда, интервальную формулу, состав нот и диаграмму аппликатуры. Аккорд можно воспроизвести и добавить в избранное.

Визуализатор грифа и гамм отображает выбранную гамму на гитарном грифе. Он помогает связать теоретический состав гаммы с расположением нот на инструменте. Пользователь может слушать гамму, выбирать тональность и режим отображения.

Тренажёр интервалов воспроизводит две ноты и предлагает определить расстояние между ними. Такой модуль направлен на развитие музыкального слуха и закрепление теоретических знаний на практике.

Построитель прогрессий формирует диатонические аккорды выбранной тональности, позволяет составлять последовательности, сохранять пользовательские пресеты и воспроизводить результат. Круг квинт показывает родственные тональности, ключевые знаки и популярные гармонические связи.

**Листинг 4**

Определение основной частоты методом FFT и HPS

```
public float DetectPitch(float[] samples, float gain = 1f)
{
    if (samples.Length < _fftSize) return 0f;

    var buf = new ComplexFloat[_fftSize];
    for (int i = 0; i < _fftSize; i++)
    {
        double w = 0.5 * (1.0 - Math.Cos(2.0 * Math.PI * i / (_fftSize - 1)));
        buf[i] = new ComplexFloat(samples[i] * gain * (float)w, 0f);
    }

    Fft.Forward(buf);

    int half = _fftSize / 2;
    float[] spec = new float[half];
    for (int i = 0; i < half; i++)
        spec[i] = buf[i].Magnitude;

    float[] hps = (float[])spec.Clone();
    for (int h = 2; h <= 5; h++)
        for (int i = 0; i < half / h; i++)
            hps[i] *= spec[i * h];

    int minBin = (int)(70f * _fftSize / _sampleRate);
    int maxBin = Math.Min((int)(1400f * _fftSize / _sampleRate), half / 5);
    int peak = minBin;
    for (int i = minBin; i < maxBin; i++)
        if (hps[i] > hps[peak]) peak = i;

    float freq = peak * _sampleRate / (float)_fftSize;
    return (freq >= 70f && freq <= 1400f) ? freq : 0f;
}
```

В листинге 4 показана логика частотного анализа. Входной сигнал умножается на окно Хэннинга, затем преобразуется с помощью FFT. Для выделения основной частоты используется Harmonic Product Spectrum, после чего пик уточняется интерполяцией.

**Листинг 5**

Обработка аудиосэмплов в движке тюнера

```
public void ProcessSamples(float[] samples, int count)
{
    for (int i = 0; i < count; i++)
    {
        _ring[_ringPos & (_fftSize - 1)] = samples[i] * Gain;
        _ringPos++;
    }
```

```
        float sum = 0f;
        for (int i = 0; i < _fftSize; i++)
            sum += _ring[i] * _ring[i];
        CurrentVolume = MathF.Sqrt(sum / _fftSize);

        if (CurrentVolume < SilenceThreshold)
        {
            CurrentNote = "—";
            return;
        }

        float[] ordered = new float[_fftSize];
        int start = _ringPos % _fftSize;
        for (int i = 0; i < _fftSize; i++)
            ordered[i] = _ring[(start + i) % _fftSize];

        float freq = _detector.DetectPitch(ordered);
        if (freq <= 0f) return;

        _smoothedFreq = _smoothedFreq * 0.3f + freq * 0.7f;
        var (note, cents) = NoteUtils.FrequencyToNote(_smoothedFreq, ReferenceA);
        CurrentNote = note;
        NoteDetected?.Invoke(CurrentNote, _smoothedFreq, cents);
    }
```

Движок тюнера работает с кольцевым буфером. Это позволяет постепенно накапливать сэмплы, вычислять громкость, отбрасывать тишину и обновлять отображаемую ноту только после стабилизации результата.

**Листинг 6**

Генерация щелчков метронома по позиции сэмплов

```
    public void ProcessBlock(float[] output, int numSamples)
    {
        if (!_isRunning || _accentClick == null || _normalClick == null)
            return;

        int samplesPerBeat = (int)((double)_sampleRate * 60.0 / _bpm);
        for (int i = 0; i < numSamples; i++)
        {
            int posInBeat = (int)(_samplePos % samplesPerBeat);
            if (posInBeat == 0)
                _currentBeat = (int)((_samplePos / samplesPerBeat) % _beatsPerMeasure);

            float[] click = (_currentBeat == 0) ? _accentClick : _normalClick;
            if (posInBeat < click.Length)
                output[i] += click[posInBeat] * _volume;
            _samplePos++;
        }
    }
```

В листинге 6 метроном не зависит от графического таймера. Позиция доли вычисляется по количеству обработанных сэмплов, поэтому щелчки остаются синхронными с аудиопотоком.

**Листинг 7**

Упрощённая структура аудиообработчика VST3-плагина

```
public override void Process()
{
    Span<double> input = _monoInput.GetAudioBuffer(0);
    Span<double> outL = _stereoOutput.GetAudioBuffer(0);
    Span<double> outR = _stereoOutput.GetAudioBuffer(1);

    for (int i = 0; i < input.Length; i++)
    {
        outL[i] = input[i];
        outR[i] = input[i];
    }

    Tuner.ProcessSamples(ConvertToFloat(input), input.Length);
    Metronome.ProcessBlock(_metroBuffer, input.Length);
    MixToStereo(_metroBuffer, outL, outR);
}
```

В VST3-режиме обработчик Process вызывается хостом для каждого аудиоблока. В нём входной сигнал передаётся на выход, копируется в тюнер и дополняется метрономом или воспроизводимыми нотами. Реальная реализация дополнительно защищена от исключений, чтобы снизить риск падения DAW.

В функциональной части приложения предусмотрены следующие пользовательские сценарии:

- выбор входного устройства и настройка гитары по микрофону или аудиоинтерфейсу;
- запуск метронома с заданным темпом и размером такта;
- поиск аккорда по тонике и типу с просмотром аппликатуры;
- отображение гаммы на грифе и её воспроизведение;
- проверка интервалов на слух;
- составление аккордовых прогрессий и сохранение пользовательских наборов.

# 3. ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ

## 3.1. Руководство пользователя

Для запуска standalone-версии пользователь распаковывает архив GuitarToolkit_DESKTOP_v.1.2.0.zip и запускает исполняемый файл приложения. После запуска в верхней панели выбирается входное аудиоустройство. Если устройство доступно, тюнер начинает получать сигнал и отображает состояние записи.

Для установки VST3-версии пользователь распаковывает архив GuitarToolkit_VST3_v.1.2.0.zip и помещает папку плагина в каталог VST3-плагинов Windows. После повторного сканирования плагинов в DAW GuitarToolkit появляется в списке эффектов и может быть добавлен на аудиодорожку.

Работа с приложением строится вокруг вкладок. Пользователь выбирает нужный инструмент слева, настраивает параметры в верхней или центральной части окна и получает результат в виде звука, диаграммы или справочной информации. Такой сценарий одинаков для standalone-режима и режима плагина.

## 3.2. Тестирование

Тестирование проекта выполнялось в двух направлениях: автоматическая проверка ядра приложения и ручная проверка пользовательских сценариев. Автоматические тесты находятся в проекте GuitarToolkit.Tests и запускаются командой dotnet test GuitarToolkit.sln --no-restore --configuration Debug.

На момент подготовки отчёта тестовый набор содержит 73 модульных теста. Они проверяют определение высоты тона, модели музыкальной теории, работу метронома, построение прогрессий и корректность базовых вычислений.

### *3.2.1. Первый тест*

Первый тест связан с проверкой алгоритма определения высоты тона и отображения результата в модуле тюнера. Для этого используются синусоидальные сигналы с известной частотой, а также сигналы с гармониками, похожие на звучание гитарной струны. Ожидаемый результат сравнивается с частотой и нотой, рассчитанной тюнером.

Практическая проверка выполнялась при выбранном входном устройстве. На вход подавался сигнал, соответствующий ноте C#. Приложение должно было определить ноту, показать частоту, отклонение в центах, состояние настройки и уровень входного сигнала.

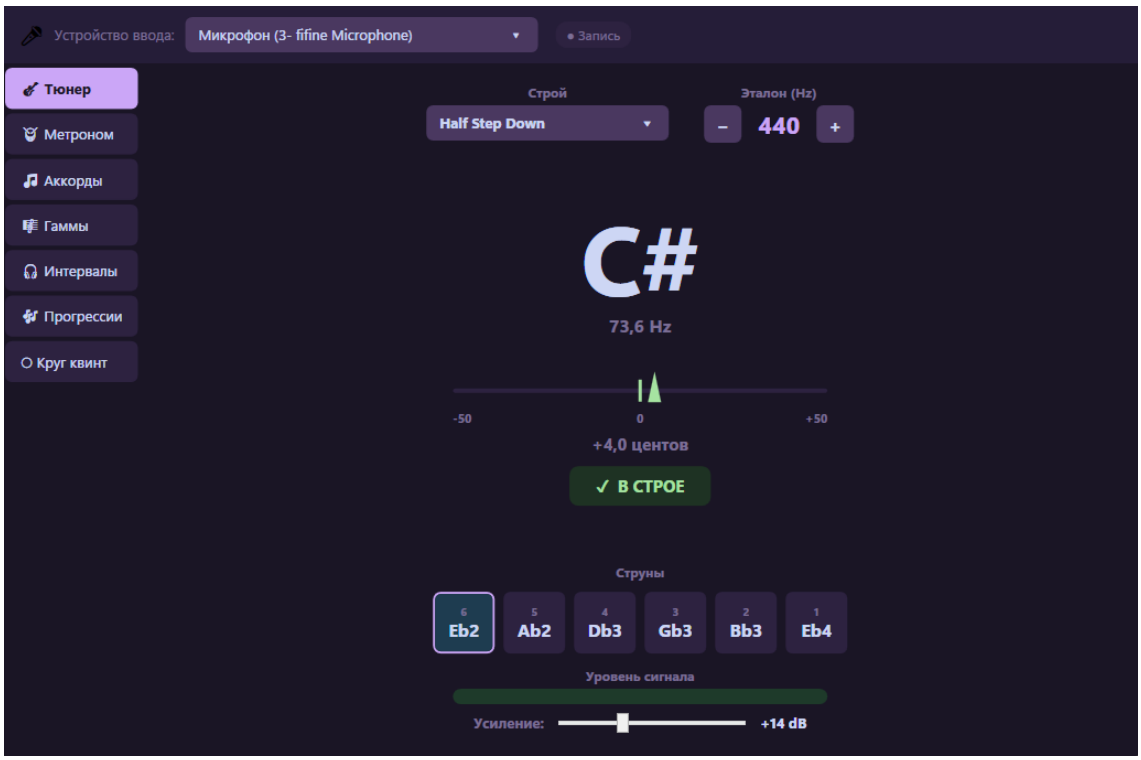


*Рис. 2. Результат проверки модуля тюнера.*

На рисунке 2 видно, что модуль тюнера определяет ноту C#, отображает частоту 73,6 Гц, отклонение +4,0 цента и состояние «В строе». Это подтверждает корректную связь между аудиоанализом, моделью нот и пользовательским отображением результата.

Данный тест важен, потому что тюнер является одной из ключевых функций приложения. При недостаточной точности пользователь не сможет

корректно настроить инструмент, поэтому проверка DSP-алгоритма является обязательной.

### *3.2.2. Второй тест*

Второй тест проверяет музыкально-теоретические модели аккордов. В ходе проверки контролируется корректность формул, наличие аппликатур, состав нот и работа избранного. Для эксперимента выбран аккорд Am, так как он имеет простую открытую аппликатуру и хорошо подходит для проверки отображения грифа.

Пользователь выбирает тонику A и тип «Минор». Приложение должно показать название аккорда, формулу 1-б3-5, состав нот A-C-E, диаграмму аппликатуры и кнопки переключения вариантов. Дополнительно проверяется возможность воспроизведения аккорда.

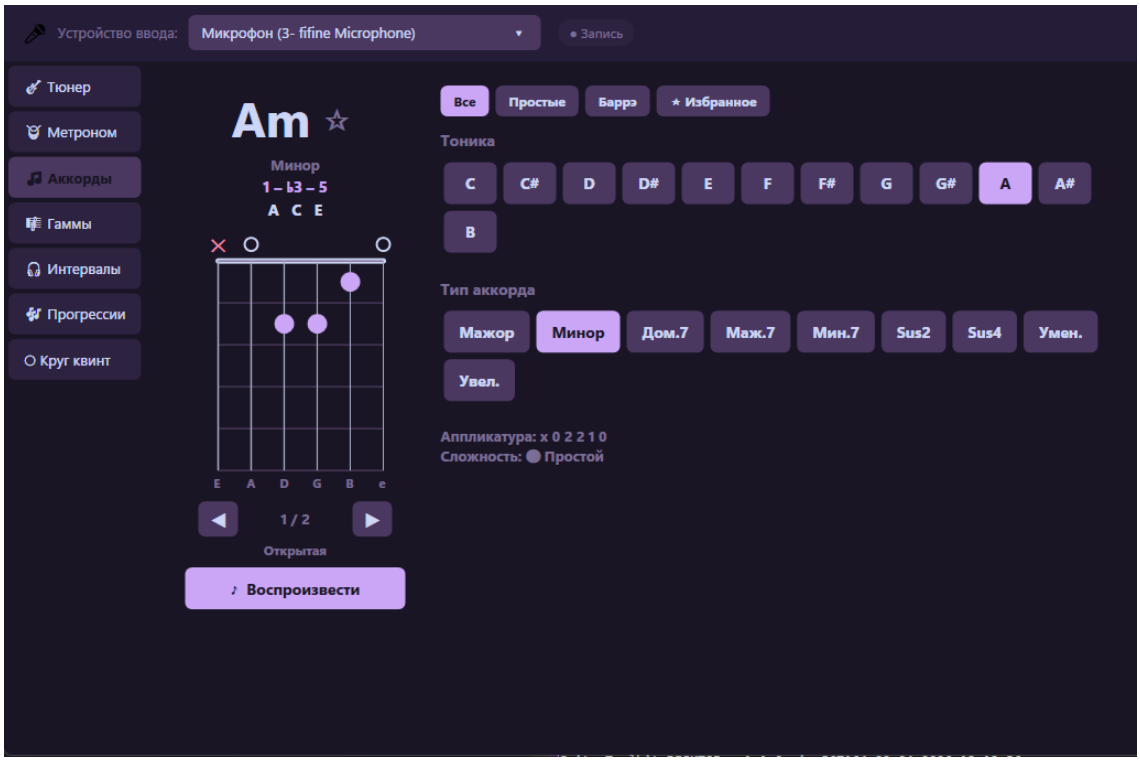


*Рис. 3. Результат проверки справочника аккордов.*

На рисунке 3 справочник отображает аккорд Am, его интервальную формулу и аппликатуру. Это подтверждает, что данные ChordDefinition корректно передаются в визуальный компонент и используются для построения диаграммы.

### *3.2.3. Третий тест*

Третий тест связан с проверкой визуализатора грифа и гамм. Для эксперимента выбрана тональность A и натуральный минор. Приложение должно вычислить состав гаммы, отобразить ноты на всех струнах и ладах, а также дать возможность воспроизвести гамму.

Проверка важна потому, что визуализатор объединяет несколько частей проекта: модель гамм, модель строя гитары, расчёт позиций нот на грифе и WPF-отображение результата. Ошибка в любой из этих частей сразу становится заметной на схеме грифа.

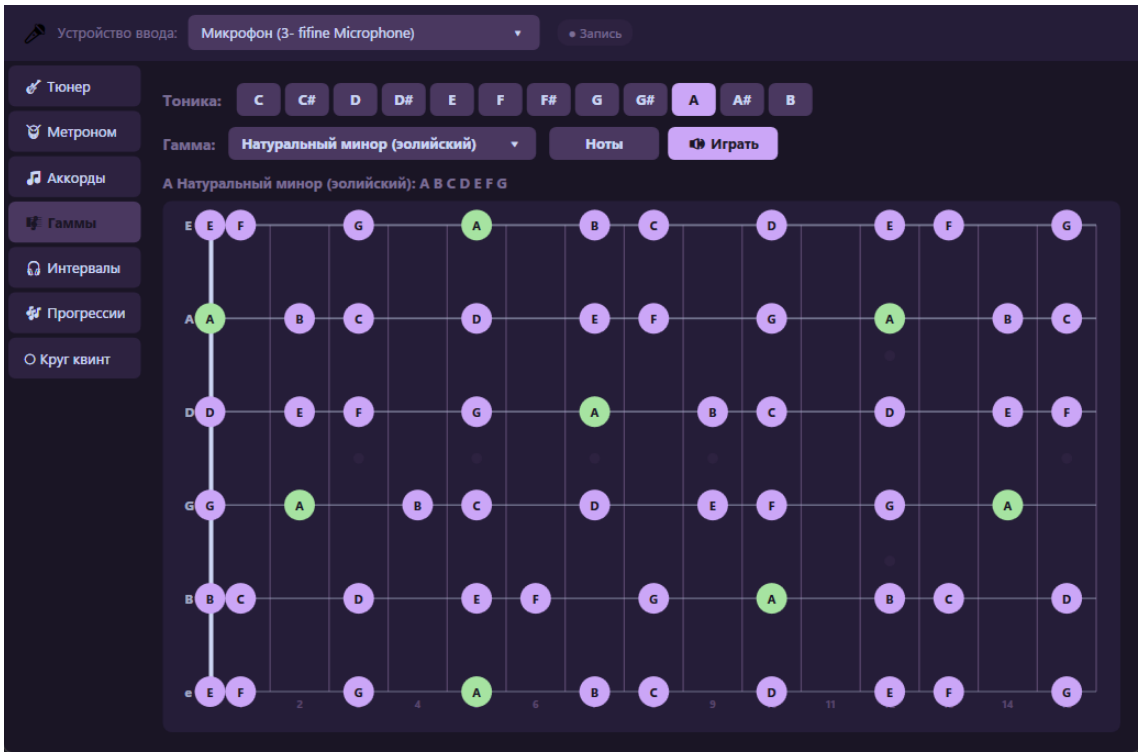


*Рис. 4. Результат проверки визуализатора грифа и гамм.*

На рисунке 4 показана гамма A натуральный минор. Ноты A, B, C, D, E, F и G отображаются на грифе, тоника выделена другим цветом. Это подтверждает корректность расчёта гаммы и её отображения на виртуальном инструменте.

В VST3-режиме дополнительно проверялась загрузка плагина в DAW, получение входного сигнала с аудиодорожки, работа тюнера и метронома. Отдельно проверялся сценарий, при котором вход записи назначается уже после добавления плагина на дорожку.

### *3.2.4. Выводы*

По результатам тестирования приложение выполняет основные функции, заявленные в техническом задании. Сборка решения проходит успешно, автоматические тесты подтверждают корректность ядра, а ручная проверка показывает работоспособность основных пользовательских сценариев.

Для повышения надёжности VST3-режима критические участки аудиообработки оставлены в консервативной реализации. Это решение принято в пользу стабильности работы в DAW-хосте, поскольку даже небольшая ошибка в аудиопотоке может привести к аварийному завершению всей DAW.

Проект подготовлен к публикации: оформлен README, добавлены скриншоты интерфейса, создан релизный скрипт, подготовлены инструкции установки, шаблоны GitHub Issues и опубликован релиз версии 1.2.0.

# ЗАКЛЮЧЕНИЕ

В ходе учебной практики был рассмотрен и оформлен проект программного продукта GuitarToolkit. В результате получено приложение, объединяющее несколько инструментов, необходимых гитаристу для настройки инструмента, тренировки ритма, изучения аккордов, гамм, интервалов и построения прогрессий.

Разработка выполнена с разделением на ядро, общий пользовательский интерфейс и две платформенные формы поставки. Такое архитектурное решение позволяет использовать одну кодовую базу как в standalone-приложении, так и в VST3-плагине.

Важной частью работы стало тестирование. Наличие 73 модульных тестов повышает надёжность ядра приложения и позволяет контролировать корректность музыкально-теоретических моделей и DSP-компонентов.

Проект подготовлен к публикации: добавлена иконка приложения, оформлен README, создан релизный скрипт, подготовлены инструкции установки, шаблоны GitHub Issues и опубликован релиз версии 1.2.0. Таким образом, поставленные задачи учебной практики можно считать выполненными.

# СПИСОК ЛИТЕРАТУРЫ

Microsoft. Документация по платформе .NET 8. URL: https://learn.microsoft.com/dotnet/

Microsoft. Документация по WPF. URL: https://learn.microsoft.com/dotnet/desktop/wpf/

AudioPlugSharp. Документация и материалы проекта. URL: https://github.com/mikeoliphant/AudioPlugSharp

NAudio. Документация библиотеки аудиообработки для .NET. URL: https://github.com/naudio/NAudio

Steinberg Media Technologies. VST3 SDK Documentation. URL: https://steinbergmedia.github.io/vst3_dev_portal/

xUnit.net. Documentation. URL: https://xunit.net/

GitHub Docs. Releases and project documentation. URL: https://docs.github.com/